

Design Justification Report

INT8 Conv2D MAC Accelerator on sky130A

ECE 410/510 — Hardware for AI and ML (HW4AI), Spring 2026 — Milestone 4

June 2026

1. Problem and Motivation

The kernel targeted for hardware acceleration is the **two-dimensional convolution (Conv2D)** that forms the compute core of a convolutional neural network (CNN). In Milestone 1, a pure-NumPy CNN was profiled on an Intel Core i5-8265U laptop CPU using cProfile. The reference network is a single Conv2D layer with a 3×3 kernel, one input channel, eight output channels, and a 16×16 input feature map (configuration `conv_layers=[(8,3,1)]`, `batch_size=32`), followed by ReLU, max-pooling, and dense layers.

The profiling identified Conv2D as the dominant kernel: the convolution forward pass (`cnn.py:128`) consumed **26.9% of total runtime**, with the `im2col` transformation and the associated matrix multiplication forming the hot path. For one batch of 32 samples the layer performs

$$\text{FLOPs} = 2 \times N \times C_{in} \times k^2 \times H_{out} \times W_{out} \times C_{out} = 2 \times 32 \times 1 \times 9 \times 16 \times 16 \times 8 = 1,179,648 \text{ FLOPs.}$$

The measured per-batch time for that layer was **4.22 ms** (cumulative time per call, 3.208 s over 760 calls), giving a measured software throughput of $1,179,648 / 0.00422 = \mathbf{0.280 \text{ GOPS}}$. Even with vectorized NumPy, this is roughly 0.7% of the CPU’s memory-bound ceiling at this kernel’s arithmetic intensity — the gap is interpreter overhead, object dispatch, and the absence of dedicated MAC hardware.

Custom hardware is justified precisely because the operation is small, regular, and repeated billions of times in CNN inference. A fixed-function MAC array eliminates instruction fetch and control overhead, processes the multiply-accumulate in dedicated silicon, and streams data deterministically. The goal of this project was to take this kernel the full distance — from RTL through physical synthesis to a manufacturable, signed-off layout on the open-source **sky130A** process using **OpenLane 2** — rather than stopping at FPGA emulation or high-level synthesis. The high-level partition is shown in Figure 1: the host quantizes and streams data to the accelerator over AXI4-Stream, the accelerator computes, and results stream back.

Figure 1. High-Level System Block Diagram

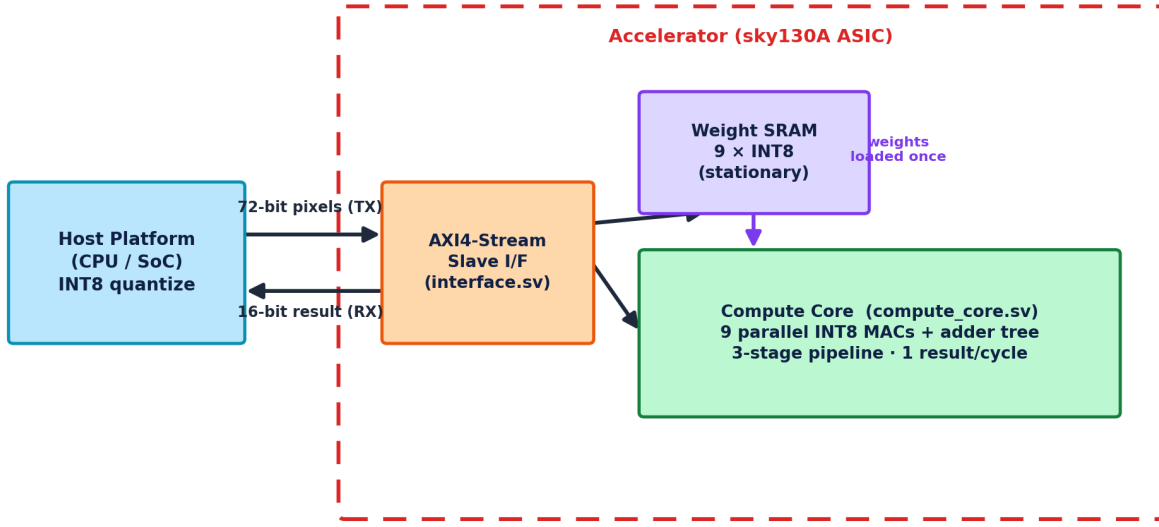


Figure 1: High-level system block diagram.

2. Roofline Analysis

The roofline model from Milestone 1 anchored every subsequent architectural decision. The i5-8265U has a peak compute of **102.4 GFLOP/s** and a peak memory bandwidth of **37.5 GB/s**, placing its ridge point at $102.4/37.5 = 2.73$ FLOP/byte. The Conv2D kernel’s arithmetic intensity was computed from the exact im2col footprint: 1,179,648 FLOPs over 1,114,688 bytes (FP64, 8 bytes per element) gives **AI = 1.058 FLOP/byte**. Because 1.058 is well to the left of the 2.73 ridge, the kernel is firmly **memory-bound** on the CPU: its theoretical ceiling is $37.5 \times 1.058 = 39.7$ GFLOP/s, and the measured 0.280 GOPS sits far below even that.

The single most important insight from the roofline is that *reducing bytes moved per operation shifts the operating point rightward, toward the compute-bound regime*. This directly motivated the move to INT8 (Section 3). Replacing 8-byte FP64 elements with 1-byte INT8 elements reduces byte traffic by 8×, multiplying arithmetic intensity by the same factor: **1.058 → 8.47 FLOP/byte**. At AI = 8.47, the operating point lies to the right of both the CPU ridge (2.73) and the hardware ridge (2.0), which means the design is **compute-bound** — its throughput is limited by how many MACs it can perform per cycle, not by how fast data arrives. That is exactly the regime a parallel MAC array is built to exploit.

Figure 2 plots the final picture against the same axes as Milestone 1. The software kernel appears at (1.058, 0.280); the final accelerator appears at the **measured** point (8.47, 1.5 GOPS), sitting on its own compute ceiling. The accelerator point reflects the measured post-synthesis value, not the Milestone 1 hypothetical design target. A second consequence visible in the figure is that both the M3 and M4 hardware rooflines have their ridge at exactly **2.0 FLOP/byte**: widening the design from one

MAC to nine scaled compute and interface bandwidth by the same 9 $\times$, preserving the balance point.

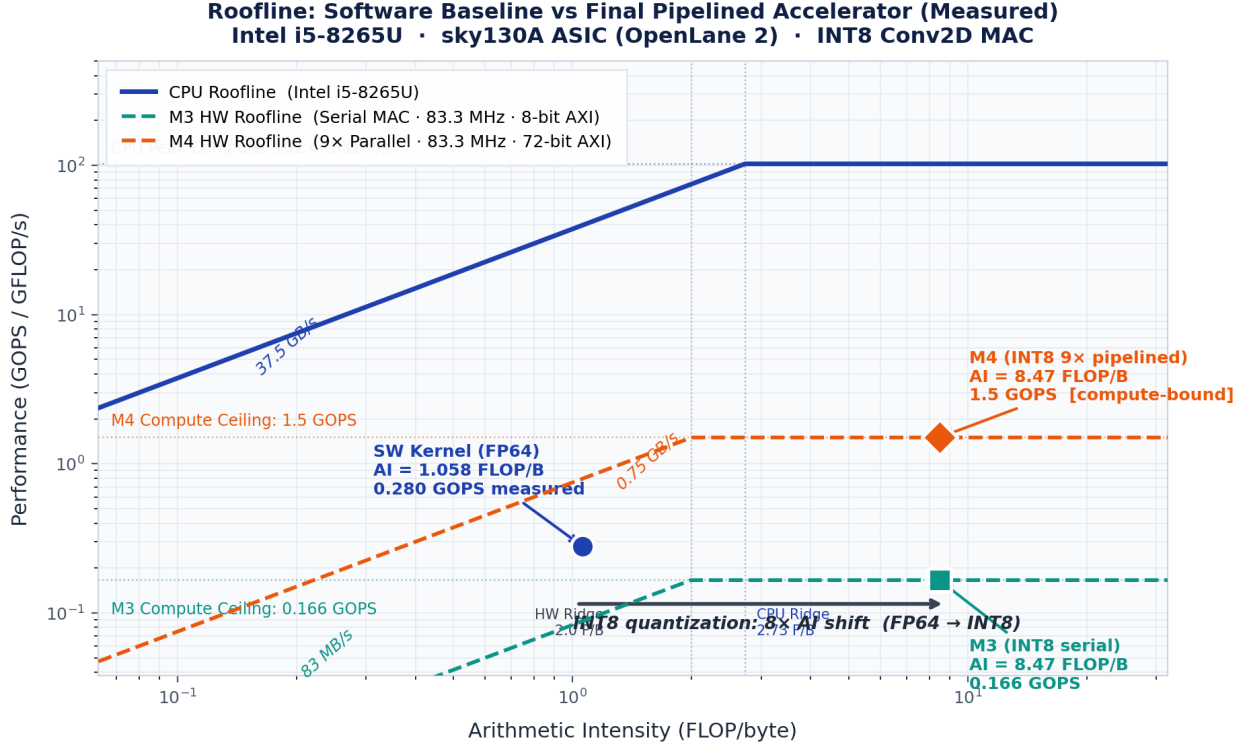


Figure 2: Roofline: software baseline versus the final pipelined accelerator (measured).

3. Precision and Data Format

The accelerator uses **8-bit signed integer (INT8)** arithmetic in place of the FP32/FP64 software baseline, a decision grounded directly in the roofline analysis of Section 2. Quantizing weights and activations from floating point to INT8 cuts byte traffic over the interface by 4 $\times$ relative to FP32 (8 $\times$ relative to the FP64 profiling baseline), which is what shifts the arithmetic intensity into the compute-bound regime. INT8 multipliers are also far smaller in silicon than floating-point units, reducing area and power.

A symmetric quantization scheme with a **scale factor of 127** maps the normalized floating-point range $[-1, 1]$ onto the signed 8-bit range $[-127, 127]$. On the host, a floating-point value is multiplied by 127 and rounded to nearest. Each 8×8 signed multiply produces a 16-bit product, and the nine products accumulate in a 16-bit datapath; the final result is dequantized on the host by dividing by 127².

The acceptability of INT8 was verified empirically in Milestone 2. An automated bench executed **100 randomized 3 $\times$ 3 convolutions** with uniform-random weights and pixels, comparing the INT8 hardware model against the FP NumPy reference. The results were a **mean absolute error of 0.004226** and a **maximum error of**

0.013897 — under 0.5% average and under 1.4% worst-case deviation. CNNs are inherently robust to small perturbations in intermediate feature maps because subsequent ReLU and pooling stages suppress them, so this error budget is comfortably acceptable for inference. The tradeoff is overwhelmingly favorable: a negligible loss of precision in exchange for $4\times$ less interface traffic, smaller multipliers, and lower power.

4. Dataflow and Architecture

The accelerator implements a **weight-stationary** dataflow. For a 3×3 convolution the nine kernel weights are loaded once into an on-chip register file and held fixed while pixel windows stream through. Weight-stationary is the natural fit here because each set of nine weights is reused across every spatial position of the output feature map, so keeping them resident eliminates repeated weight fetches and confines external traffic to the streaming pixels.

The compute core, shown in Figure 3, evolved across milestones. The Milestone 3 baseline (`compute_core.sv`) was a **single serial MAC**: one 8×8 multiplier feeding a 16-bit accumulator, consuming one pixel per cycle and requiring nine cycles per output. Milestone 4 widened this into a **nine-wide parallel MAC array**. A 72-bit AXI input delivers all nine pixels of a 3×3 window in a single transfer; nine parallel 8×8 signed multipliers compute the nine products simultaneously, and a balanced **adder tree** sums them. The finite-state machine has two states: `LOAD_WEIGHTS`, which captures the nine weights into the stationary register over the 72-bit bus, and `COMPUTE`, which streams pixel windows and emits one MAC result per cycle.

The decisive Milestone 4 refinement was **pipelining**. In the initial parallel version the nine multipliers and the full adder tree formed a single combinational cloud in one clock period, which limited frequency and — as Section 9 explains — burned excessive glitch power. The final design inserts pipeline registers to form a **three-stage pipeline**: Stage 1 captures the incoming pixel data and the stationary weights from the weight cache into operand registers (`data_reg`, `weight_reg`); Stage 2 computes the nine signed 8×8 products and registers them (`mult_reg`); Stage 3 passes the nine products through a balanced, four-level combinational adder tree and latches the final sum into the output register (`mac_out`). No single combinational stage carries the full multiply-plus-sum delay. Pipeline latency is three cycles; sustained throughput is **one output per cycle**. This is the structure that produced the final benchmark numbers.

5. Hardware Interface

The chosen interface is **AXI4-Stream**, implemented as a slave on the accelerator (`interface.sv`). The selection was made in Milestone 1 against the full set of candidates. SPI and I²C were rejected as far too low-bandwidth (tens of Mbit/s) for a streaming inference datapath. AXI4-Lite is a register/control interface with handshake overhead unsuited to continuous data flow — it is appropriate for the control plane, paired with AXI4-Stream for data, which is the standard practice. PCIe (Gen3/4) was rejected

Figure 2. Compute-Core Dataflow — Weight-Stationary, 3-Stage Pipelined 9-Wide INT8 MAC

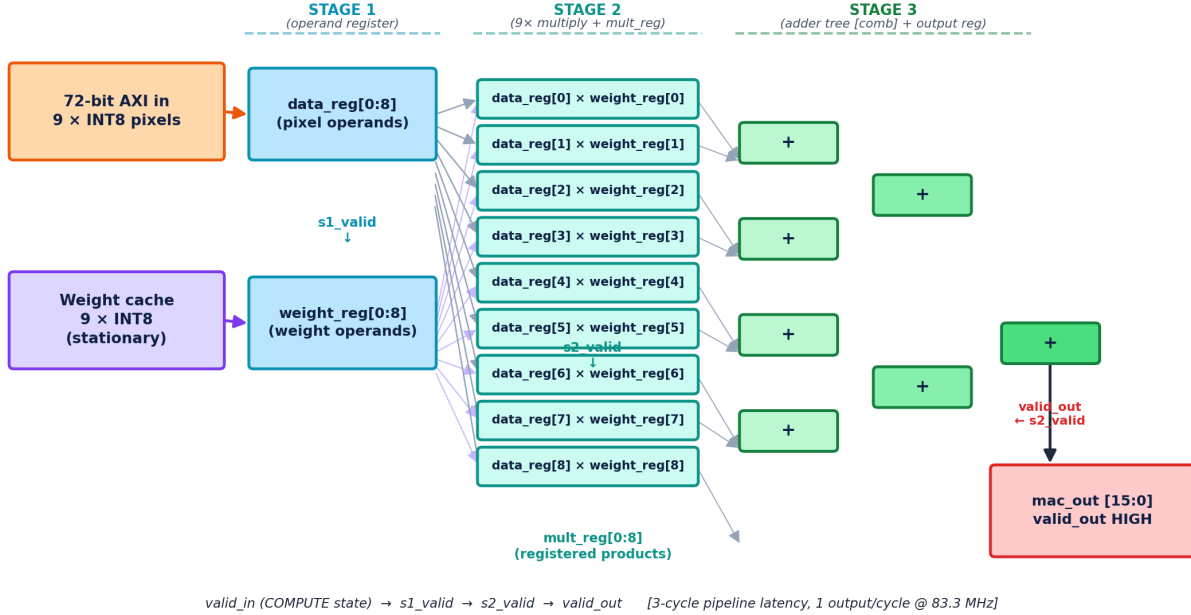


Figure 3: Compute-core dataflow: nine-wide parallel MAC with pipelined adder tree.

because it requires a PCIe endpoint controller IP and significant implementation complexity, and targets data-center boards rather than an embedded SoC. UCIE was rejected because it is a chiplet-to-chiplet interconnect requiring a multi-die physical design beyond this project’s scope. AXI4-Stream is the ARM AMBA standard for uni-directional streaming: deterministic, low-latency, configurable in width, and natively supported in every FPGA SoC (e.g., Xilinx Zynq), making it the correct match for a continuous inference pipeline.

The data bus is **72 bits wide**, carrying nine INT8 pixels per transfer to feed the nine parallel multipliers; results return as 16-bit values. At the final operating frequency of 83.3 MHz the interface delivers $9 \text{ bytes} \times 83.3 \text{ MHz} = 750 \text{ MB/s} = 0.75 \text{ GB/s}$ of sustained pixel bandwidth. The design is **not interface-bound**: the streaming intensity at the bus is $18 \text{ ops}/9 \text{ bytes} = 2.0 \text{ ops/byte}$, which is exactly the hardware ridge point, and the algorithm’s arithmetic intensity (8.47 FLOP/byte) sits far to the right of it. In practice this means the bus supplies one full window every cycle and the nine MACs never stall waiting for data — the compute, not the interface, is the limiting factor, which is the intended balance.

6. Verification

Functional correctness was established by self-checking testbenches (`tb_top.sv`) driving the integrated top module, following the same PASS/FAIL contract used in Milestones 2 and 3, and confirmed at two levels: the INT8 numerical model (100 randomized convolutions, Section 3) and the RTL itself.

Two independent RTL test vectors were used, chosen to cover complementary failure modes. **Test 1** applied unit weights $\{1, 1, 1, 1, 1, 1, 1, 1, 1\}$ and produced the result **450**, confirming the data path, the FSM transition from LOAD_WEIGHTS to COMPUTE, the accumulation through the adder tree, the pipeline timing (valid_out asserting on the correct cycle with stable data), and clean reset behavior. Test 1 alone, however, does not stress the multiplier: with every weight equal to one, a multiplier that ignored its weight and passed the pixel through would still produce 450. **Test 2** therefore applied mixed weights including negative values $(-3, -4, -2, -1, \dots)$ and produced exactly **150** — a result that is only correct if the signed 8×8 multiplication is implemented properly and the negative partial products subtract correctly within the adder tree. Had sign handling been broken, the result would have been grossly wrong. Together the two vectors verify the accumulation path and the signed-arithmetic path independently. The annotated end-to-end transaction is shown in Figure 4 and committed at `sim/final_waveform.png`; the passing run is recorded in `sim/final_run.log`.

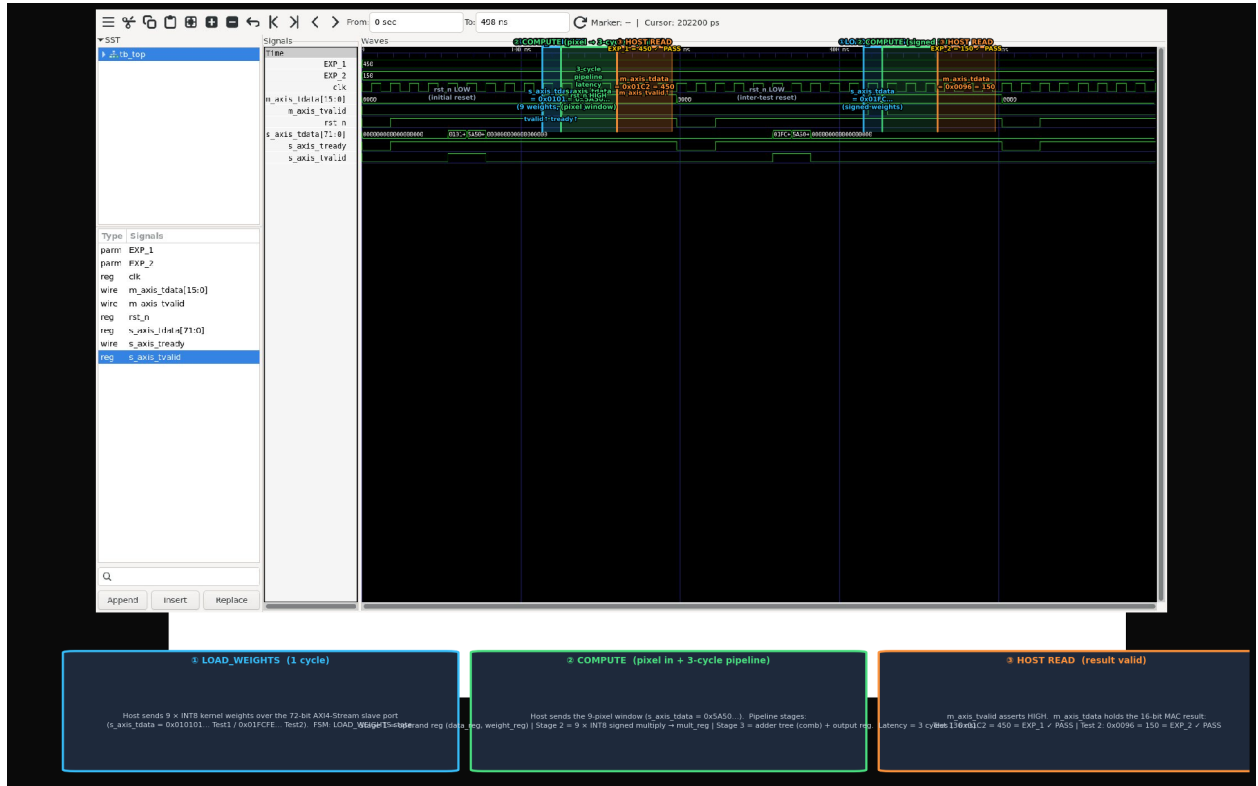


Figure 4: Annotated end-to-end simulation waveform.

7. Synthesis Results

The final design was pushed through **OpenLane 2 (sky130A)** to a fully signed-off layout. The exact configuration is committed in `config.json` (clock period 12 ns, source list, constraints) and the complete tool transcript in `openlane_run.log`. Numbers below are drawn from the committed `timing_report.txt`, `area_report.txt`, and `power_report.txt`.

Timing. The design closes at a **12 ns clock period (83.3 MHz)** with positive slack at every one of the nine PVT corners. The worst-case setup slack is **+1.62 ns** at the `max_ss_100C_1v60` corner (slow process, 100 °C, 1.60 V, maximum parasitics); hold slack is positive everywhere (worst +0.145 ns); and there are **zero setup, hold, max-slew, and max-capacitance violations** across all corners. The +1.62 ns of remaining slack at the worst corner implies roughly 96 MHz of headroom if pushed.

Area. Total chip area is **51,147 μm^2** across **4,432 standard cells**, of which 23.3% (11,903 μm^2) is sequential. The dominant area contributor is the combinational arithmetic — the nine 8×8 multipliers and the adder tree — reflected in the cell mix being dominated by XNOR/XOR cells (509 `xnor2` and 268 `xor2`, the carry-save and product logic) plus 453 `df_rtp` flip-flops, the latter being the pipeline and accumulator registers.

Power. At the nominal `nom_tt_025C_1v80` corner the post-route power estimate (OpenROAD `report_power`, default switching activity) is **21.6 mW total**, split 82.3% combinational, 10.3% sequential, 7.4% clock; by type the split is 49.3% internal and 50.7% switching, with negligible leakage (33 nW, expected for 130 nm). The combinational arithmetic again dominates, consistent with the area breakdown.

Metric	M3 (serial)	M4 (pipelined)	M4 / M3
Throughput	0.166 GOPS	1.5 GOPS	9.0×
Clock	83.3 MHz	83.3 MHz	1×
Area	6,596 μm^2	51,147 μm^2	7.8×
Power (nom_tt)	2.64 mW	21.6 mW	8.2×
Energy efficiency	62.9 GOPS/W	69.3 GOPS/W	1.10×
Std cells	569	4,432	7.8×
Signoff	clean	clean (9 corners)	—

8. Benchmark Results

The accelerator is benchmarked against the Milestone 1 software baseline using the same workload — the Conv2D layer for one batch of 32 samples (1,179,648 operations). Raw values are committed in `benchmark_data.csv`.

Throughput. Accelerator throughput is computed from the design’s known rate (9 MACs/cycle = 18 ops/cycle) and the post-synthesis frequency: $18 \times 83.3 \text{ MHz} = \mathbf{1.5 \text{ GOPS}}$. Equivalently, the batch’s 1,179,648 operations complete in 65,536 cycles (one output per cycle after the 3-cycle pipeline fill), or **0.786 ms** at 83.3 MHz.

Speedup. Using the checklist definition, speedup is the ratio of baseline time to accelerator time: the M1 baseline time is **4.22 ms** per batch and the M4 accelerator time is **0.786 ms** per batch, giving a **5.4× speedup**. This matches the throughput ratio $1.5/0.280 = 5.4\times$. The comparison is honest and apples-to-apples: it uses the *measured* vectorized-NumPy baseline, not a deliberately naive Python loop, and the operation counts are identical on both sides.

Energy (optional). Multiplying the synthesis power estimate by accelerator runtime gives $21.6 \text{ mW} \times 0.786 \text{ ms} = \mathbf{17 \mu\text{J}}$ per batch. Assuming the i5-8265U draws on the

order of its 15 W TDP during the compute, the software energy is approximately $15\text{ W} \times 4.22\text{ ms} \approx 63\text{ mJ}$ per batch — roughly a **3,700× energy reduction**. This figure is sensitive to the assumed CPU power and should be read as order-of-magnitude; even so, the energy advantage of dedicated silicon is the dominant practical benefit, since the accelerator delivers higher throughput in $51,147\text{ }\mu\text{m}^2$ versus an entire CPU die.

Gap between measured and theoretical. There is no efficiency gap on the hardware side: the accelerator sustains its full compute ceiling (1.5 GOPS at 18 ops/cycle), sitting exactly on its roofline. The large remaining gap to the CPU’s *peak* compute (102.4 GFLOP/s) is expected and is not a defect — it reflects deliberate area minimization, and it closes linearly by tiling: N instances of this core yield $N \times 1.5$ GOPS, which is the path toward the original Milestone 1 design target.

9. What Did Not Work

Several attempts failed before the final clean result, and each produced a concrete lesson.

The 100 MHz target failed in Milestone 3. The first synthesis constrained the clock to 10 ns (100 MHz). The unpipelined critical path — AXI input registers through the multiplier and a linear adder chain into the accumulator — exceeded that, reporting a worst negative slack of -0.1509 ns at the `max_ss` corner. The fix at that stage was to relax the constraint to 12 ns (83.3 MHz), trading 17% of clock frequency for a guaranteed-stable baseline. The lesson — that a dense combinational arithmetic path will not meet an aggressive period without restructuring — set up the Milestone 4 pipelining work.

A Docker pathing failure blocked the first M3 run. OpenLane could not read `compute_core.sv` because Docker’s filesystem isolation broke the relative path outside the mounted directory. Mounting the repository root resolved it. The lesson was practical: the build environment must mount the full source tree the tool expects.

The first parallel (single-register) M4 was both violation-prone and power-inefficient. Widening to nine MACs at 50 MHz left a setup violation at `max_ss` (-0.196 ns) and, more stubbornly, persistent max-slew and max-capacitance violations on the 72-bit AXI bus across the slow-slow corners — the wide bus presents high fan-out and net capacitance. More surprisingly, that version measured **24.5 GOPS/W**, worse than the serial M3 (62.9 GOPS/W): nine multipliers switching in one combinational cloud burned glitch power, with combinational logic at 97.8% of total. Clearing the slew/capacitance violations required tightening fan-out constraints and letting the resizer insert buffers on the wide bus — a genuinely stubborn fight that took several iterations.

Pipelining was the resolution, and it over-delivered. Inserting the three-stage pipeline shortened the critical path enough to *raise* the clock from 50 MHz back to 83.3 MHz, and the pipeline registers latch stable values at each stage so glitches cannot propagate through the adder tree. Combinational power consequently fell from 35.9 mW to 17.8 mW even as frequency rose 67%, cutting total power from 36.7 mW to 21.6 mW and lifting efficiency to 69.3 GOPS/W — now *better* than M3. The

lesson is that pipelining is not only a frequency technique but a power technique, because it suppresses glitch propagation.

A verification coverage gap was caught and closed. The initial single test vector used unit weights, which cannot detect a multiplier that ignores its weight. Recognizing this, a second vector with mixed signed weights was added (Section 6), making the proof of signed multiplication airtight. The lesson: a passing test is not the same as a covering test, and the test vector must be chosen to actually exercise the logic under question.

If continued, the next steps would be tiling multiple cores behind a shared AXI interconnect to scale throughput, and replacing the default switching-activity assumption in the power flow with a VCD-driven estimate from the actual stimulus for a more accurate power figure.